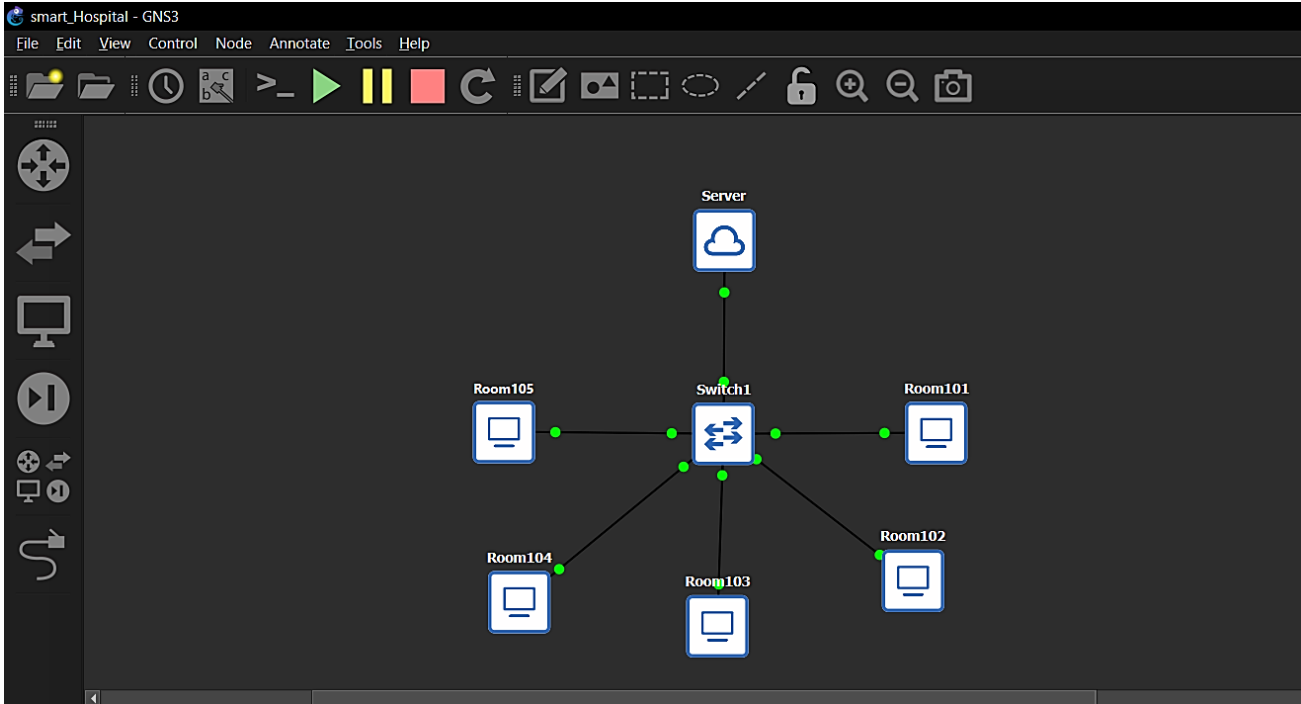


1. المقدمة

يقدم هذا المشروع تصميم وتنفيذ نظام "Enhanced Smart Hospital IoT Monitoring System" لمحاكاة المراقبة البيئية اللحظية لعدة غرف داخل مستشفى ذكي. يعتمد النظام على بنية إنترنت الأشياء (IoT) موزعة، باستخدام بروتوكول MQTT خفيف الوزن للاتصال، ومنصة Node-RED لمعالجة البيانات وعرضها عبر لوحة تحكم (Dashboard) تفاعلية.

تطور المشروع من مجرد نظام مراقبة أولي إلى نظام متكامل يحاكي بيئة الإنتاج الفعلي (Production-Level)، حيث تم دمج معمارية الثقة المكدومة (Zero Trust Architecture) وتطبيق قوائم التحكم في الوصول (ACL) لضمان أمن البيانات. بالإضافة إلى ذلك، تم دمج ميزات تسجيل البيانات (Data Logging) والتحليل الشبكي العميق لضمان موثوقية النظام واستمراريته.

تصميم الشبكة (Network Topology) من برنامج GNS3، يوضح الغرف، السيرفر (Broker)، ولوحة التحكم.

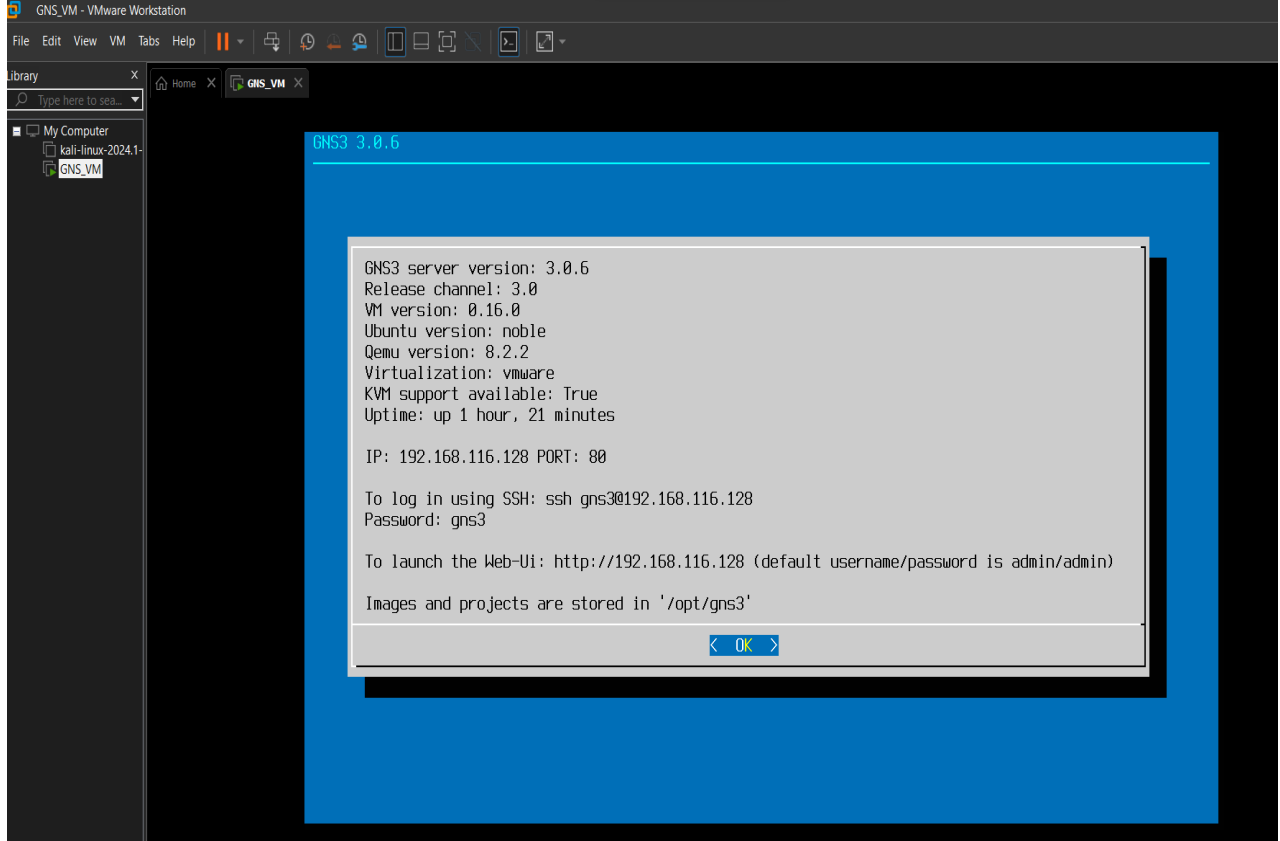


شكل (1): البنية المعمارية والشبكية لنظام المستشفى الذكي.

2. بيئة التنفيذ Implementation Environment

تم تنفيذ المشروع داخل VMware Virtual Machine لمحاكاة بيئة تشغيل معزولة وآمنة، مما يسمح باختبار النظام دون التأثير على النظام الأساسي.

VMware Virtual Machine



3. أهداف المشروع

يهدف المشروع بشكل رئيسي إلى تحقيق ما يلي:

- محاكاة مراقبة لحظية ومستمرة لعدة غرف (5 غرف كنموذج) داخل المستشفى.
- تطبيق بروتوكول MQTT لربط عُقد المستشعرات (Edge Sensor Nodes) مع نظام المعالجة المركزي.
- عرض البيانات الحية والتحليلات الزمنية (Line Charts) باستخدام Node-RED Dashboard.
- تطبيق معمارية أمنية متقدمة مبنية على مفهوم (Zero Trust) لمنع الهجمات السيبرانية والتدخل بين الغرف.

- تحليل تدفق البيانات على مستوى الشبكة (Network Packet Analysis) للتحقق من كفاءة بروتوكولات الحماية.
- أرشفة البيانات تاريخياً (Data Logging) في ملفات CSV لتسهيل التحليل الإحصائي للبيانات الطبية.

4. البنية المعمارية للنظام System Architecture

يعتمد النظام على خمس طبقات هندسية رئيسية تضمن تدفق البيانات بأمان وكفاءة:

1. طبقة المستشعرات (Sensor Simulation Layer): عُقد مبرمجة بلغة بايثون لتوليد بيانات الغرف.
2. طبقة الاتصال (Communication Layer): وسيط نقل البيانات المركزي (MQTT Broker – Mosquitto).
3. طبقة الأمن والهوية (Identity Layer & Security): نظام المصادقة وقوائم التحكم في الوصول (ACL).
4. طبقة المعالجة والتخزين (Storage Layer & Processing): منصة Node-RED وأرشفة البيانات.
5. طبقة العرض (Visualization Layer): لوحات التحكم اللحظية والرسوم البيانية للمستخدم النهائي.

5. التقنيات والبروتوكولات المستخدمة

أولاً: لغة Python وعُقد المستشعرات (Edge Nodes)

تم استخدام لغة Python لمحاكاة خمس غرف مستقلة. تعتمد هذه العُقد على واجهة برمجة التطبيقات Eclipse Paho MQTT API لإنشاء عملاء (Clients) مستقلين. يمتلك كل عميل هوية فريدة وكلمة مرور خاصة به، ويقوم بما يلي:

- الاتصال الآمن بالوسيط (Broker).
- توليد قيم حرارة (Temperature) وحالات إنذار دخان (Smoke Detection).

- إرسال البيانات بصيغة JSON كل أربع ثوانٍ عبر مسار (Topic) مخصص لكل غرفة.

مثال على هيكل البيانات المرسل (JSON Payload):

JSON

```
{ "node_id": "Room101",
  "temperature": 23.45,
  "smoke_alert": false,
  "timestamp": "2026-03-07 00:45:10" }
```

برنامج VS Code توضح كود البايثون وهو يعمل، مع ظهور رسائل الاتصال الآمن وإرسال البيانات.

The screenshot shows a VS Code editor with a Python script and its terminal output. The script is designed to connect to an MQTT broker, log in, and then publish fake fire alerts to a specific topic. The terminal output shows the successful execution of the script, with messages being published to the 'hospital/Room101/sensors' topic at regular intervals.

```
101_Edge_Nodes_02 / Hack_test.py / ...
23 print("-" * 60)
24 print(f"[1] Logging in legitimately using {HACKER_ROOM} credentials...")
25 client.connect("127.0.0.1", 1883)
26 client.loop_start()
27 time.sleep(2)
28
29 print(f"[2] Attempting to send a fake fire alert to {TARGET_TOPIC}...")
30 # Attempt to publish (qos=1 forces the broker to acknowledge)
31 client.publish(TARGET_TOPIC, FAKE_PAYLOAD, qos=1)
32
33 time.sleep(3)
34 print("-" * 60)
```

Terminal Output:

```
08:55"}
Published to hospital/Room101/sensors: {"node_id": "Room101", "temperature": 21.91, "smoke_alert": false, "timestamp": "2026-03-06 22:08:55"}
08:59"}
Published to hospital/Room102/sensors: {"node_id": "Room102", "temperature": 24.76, "smoke_alert": false, "timestamp": "2026-03-06 22:08:59"}
08:59"}
Published to hospital/Room103/sensors: {"node_id": "Room103", "temperature": 20.25, "smoke_alert": false, "timestamp": "2026-03-06 22:08:59"}
08:59"}
Published to hospital/Room104/sensors: {"node_id": "Room104", "temperature": 20.82, "smoke_alert": false, "timestamp": "2026-03-06 22:08:59"}
08:59"}
Published to hospital/Room105/sensors: {"node_id": "Room105", "temperature": 22.06, "smoke_alert": true, "timestamp": "2026-03-06 22:08:59"}
09:03"}
Published to hospital/Room101/sensors: {"node_id": "Room101", "temperature": 21.02, "smoke_alert": false, "timestamp": "2026-03-06 22:09:03"}
09:03"}
Published to hospital/Room102/sensors: {"node_id": "Room102", "temperature": 23.49, "smoke_alert": false, "timestamp": "2026-03-06 22:09:03"}
09:03"}
Published to hospital/Room103/sensors: {"node_id": "Room103", "temperature": 23.52, "smoke_alert": false, "timestamp": "2026-03-06 22:09:03"}
09:03"}
Published to hospital/Room104/sensors: {"node_id": "Room104", "temperature": 23.3, "smoke_alert": false, "timestamp": "2026-03-06 22:09:03"}
09:03"}
Published to hospital/Room105/sensors: {"node_id": "Room105", "temperature": 21.82, "smoke_alert": false, "timestamp": "2026-03-06 22:09:03"}
09:03"}
Published to hospital/Room101/sensors: {"node_id": "Room101", "temperature": 21.71, "smoke_alert": false, "timestamp": "2026-03-06 22:09:07"}
09:07"}
Published to hospital/Room102/sensors: {"node_id": "Room102", "temperature": 20.67, "smoke_alert": true, "timestamp": "2026-03-06 22:09:07"}
09:07"}
Published to hospital/Room103/sensors: {"node_id": "Room103", "temperature": 23.51, "smoke_alert": false, "timestamp": "2026-03-06 22:09:07"}
09:07"}
Published to hospital/Room104/sensors: {"node_id": "Room104", "temperature": 22.39, "smoke_alert": true, "timestamp": "2026-03-06 22:09:07"}
09:07"}
Published to hospital/Room105/sensors: {"node_id": "Room105", "temperature": 23.39, "smoke_alert": false, "timestamp": "2026-03-06 22:09:07"}
```

شكل (2): تشغيل عُقد المستشعرات (Python Nodes) وتدفق بيانات JSON.

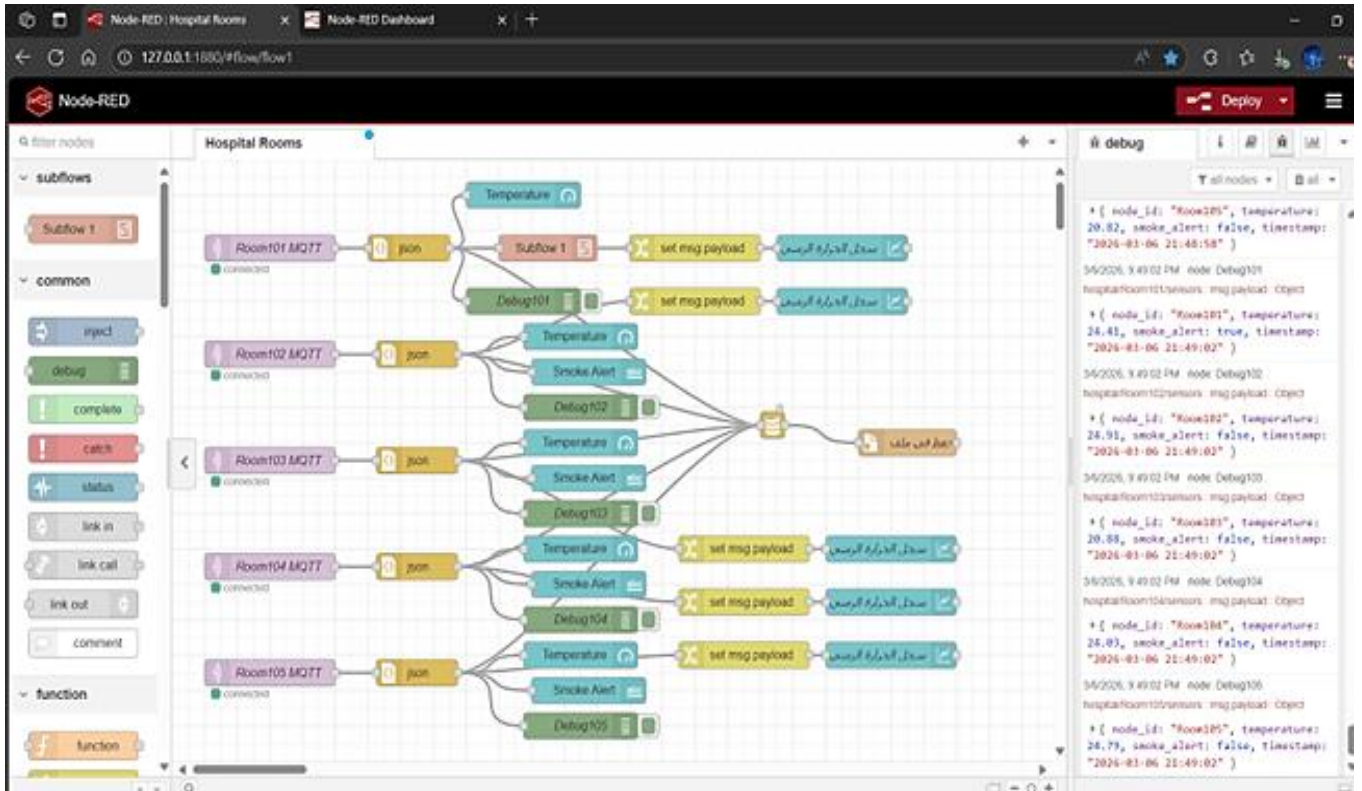
ثانياً: بروتوكول MQTT و Mosquitto Broker

تم اختيار بروتوكول MQTT لكونه خفيف الوزن (Lightweight) ومثاليًا لبيئات إنترنت الأشياء. تم إعداد خادم Mosquitto محلياً (127.0.0.1:1883) ليعمل كوسيط يستقبل الرسائل من العُقد ويعيد توجيهها إلى المشتركين.

ثالثاً: منصة Node-RED (للمعالجة والمراقبة)

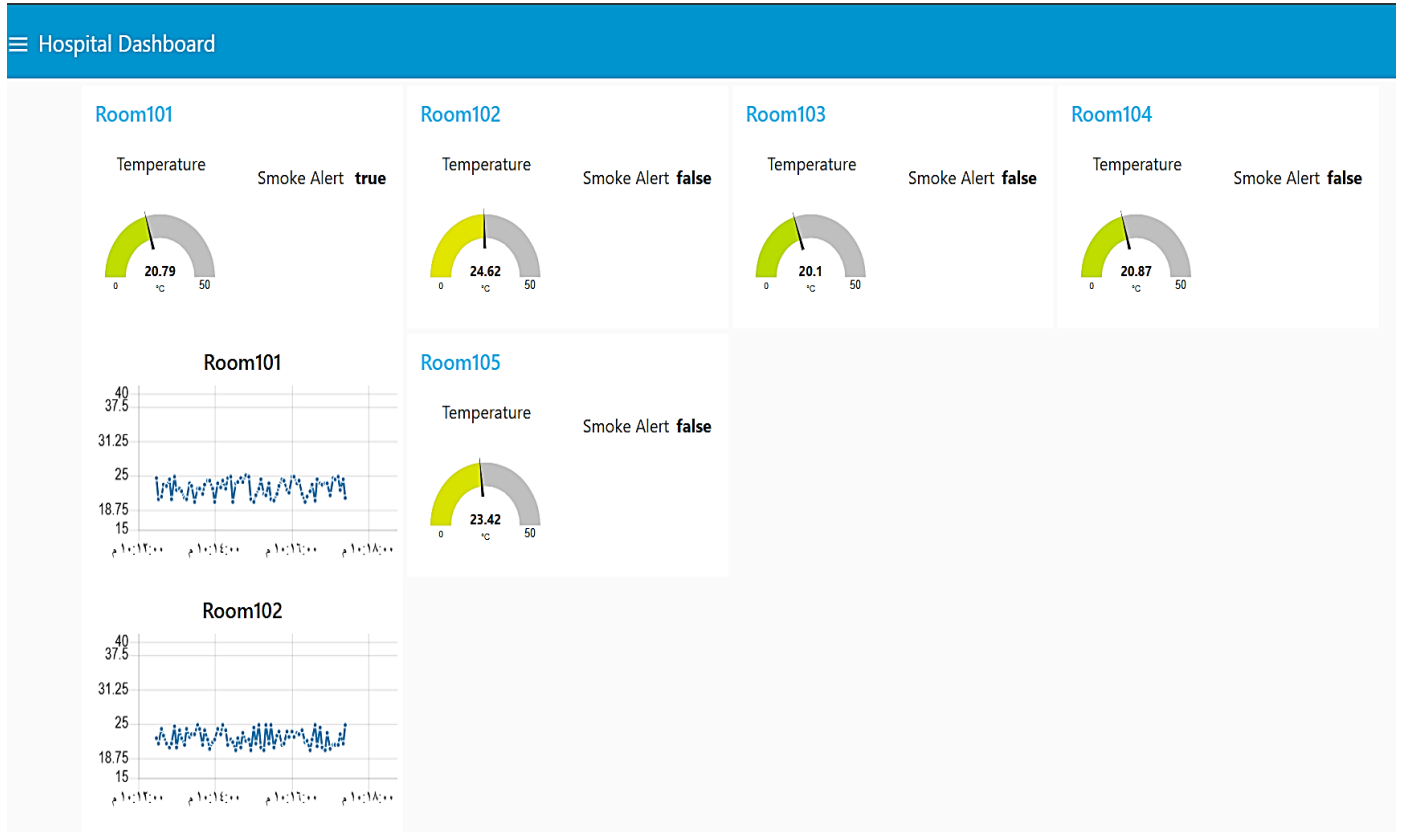
تعمل Node-RED كخرفة تحكم مركزية للبيانات. تتضمن هندسة مسارات البيانات (Flows) ما يلي:

- استقبال البيانات عبر نود MQTT In.
- فك التشفير عبر نود JSON وتصفية البيانات باستخدام Change Nodes.
- عرض البيانات لحظياً عبر العدادات (Gauges).
- رسم منحنيات التغير الزمني (Line Charts) للحرارة.
- تصدير البيانات أوتوماتيكياً (Data Logging) إلى ملف hospital_log.csv.
- مساحة العمل (Flow) توضح النودات والأسلاك ونافذة الـ Debug.



شكل (3): مسار معالجة البيانات وإعدادات التصدير (CSV) في منصة Node-RED.

لوحة التحكم (Dashboard) تظهر فيها العدادات، النصوص، والرسوم البيانية.



شكل (4): لوحة التحكم التفاعلية اللحظية (Real-time Dashboard) للمستشفى.

6. الأمن السيبراني ومعمارية الثقة المكدومة (Zero Trust Architecture)

تم التخلي كلياً عن مفهوم "الثقة الافتراضية" (Implicit Trust) في الشبكة الداخلية للمستشفى، وتطبيق المعايير التالية:

1. المصادقة الصارمة (Authentication): تم إيقاف الدخول المجهول (allow_anonymous false)، وإلزام كل جهاز بتقديم اسم مستخدم وكلمة مرور مشفرة للاتصال.
2. قوائم التحكم في الوصول (ACL - Access Control Lists): تم إنشاء قواعد وصول صارمة تمنع التداخل بين العقد. يُسمح للغرفة 101 بالكتابة في مسار hospital/Room101/sensors فقط، ولا يُسمح لها بالوصول لأي مسار آخر.

ملف acl.txt يوضح الصلاحيات المخصصة لكل غرفة.

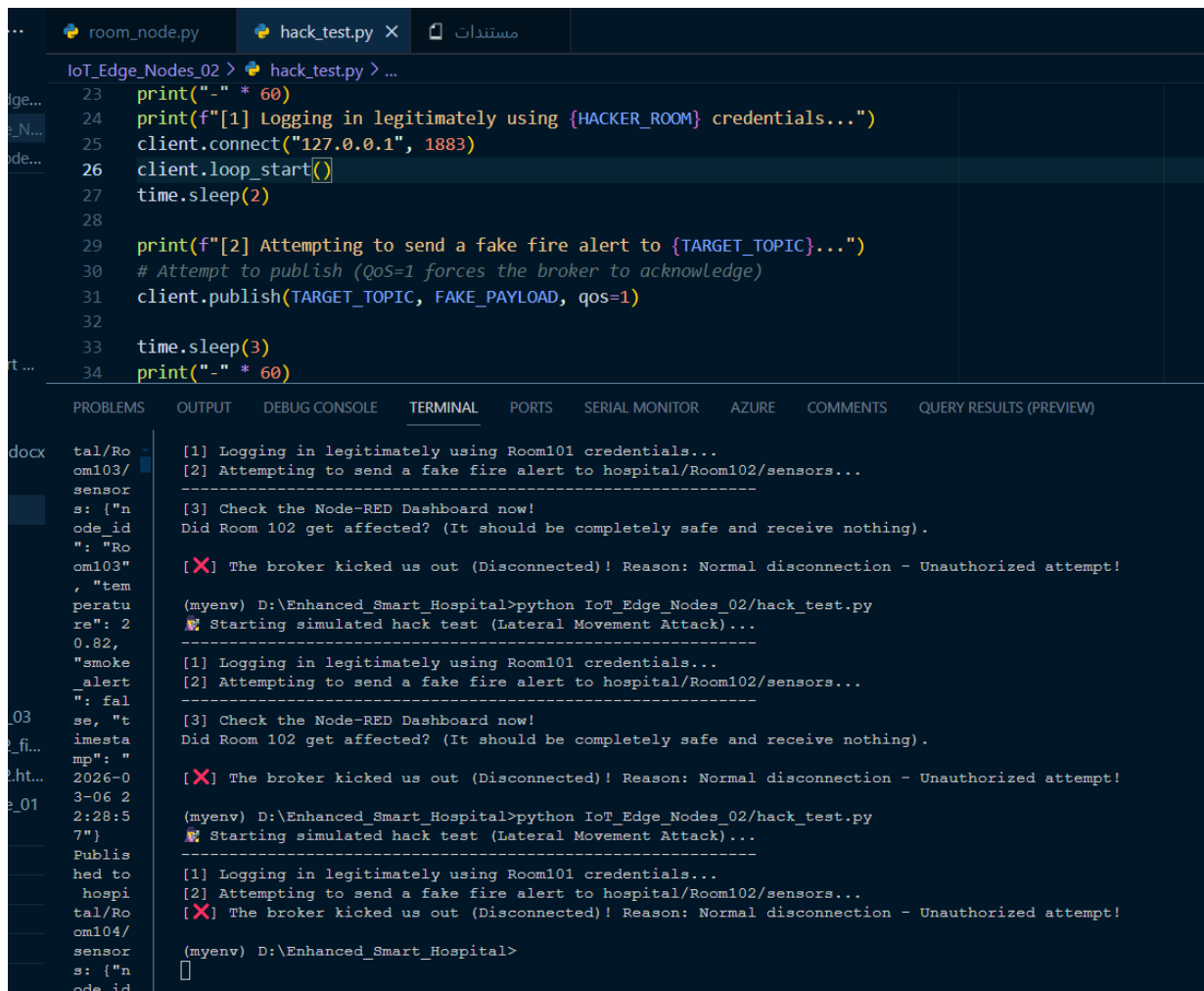
```
1 # له صلاحية قراءة كل مسارات المستشفى - (Node-RED) مدير النظام
2 user Admin
3 topic read hospital/#
4
5 # الغرفة 101 - صلاحية الكتابة في مسارها فقط
6 user Room101
7 topic write hospital/Room101/sensors
8
9 # الغرفة 102
10 user Room102
11 topic write hospital/Room102/sensors
12
13 # الغرفة 103
14 user Room103
15 topic write hospital/Room103/sensors
16
17 # الغرفة 104
18 user Room104
19 topic write hospital/Room104/sensors
20
21 # الغرفة 105
22 user Room105
23 topic write hospital/Room105/sensors
```

شكل (5): إعدادات قوائم التحكم في الوصول لتطبيق الثقة المدمومة.

7. اختبار الاختراق الوهمي (Penetration Testing):

لإثبات كفاءة النظام، تمت محاكاة هجوم "الحركة الجانبية" (Lateral Movement)، حيث حاول سكربت يمتلك صلاحيات الغرفة 101 إرسال إنذار حريق مزيف للغرفة 102. تصدى الخادم للهجوم بنجاح، ورفض الرسالة، وقام بقطع الاتصال فوراً مع العميل المخترق.

Terminal أثناء تشغيل hack_test.py توضح رسالة الطرد [X] .The broker kicked us out



```
IoT_Edge_Nodes_02 > hack_test.py > ...
23 print("-" * 60)
24 print(f"[1] Logging in legitimately using {HACKER_ROOM} credentials...")
25 client.connect("127.0.0.1", 1883)
26 client.loop_start()
27 time.sleep(2)
28
29 print(f"[2] Attempting to send a fake fire alert to {TARGET_TOPIC}...")
30 # Attempt to publish (QoS=1 forces the broker to acknowledge)
31 client.publish(TARGET_TOPIC, FAKE_PAYLOAD, qos=1)
32
33 time.sleep(3)
34 print("-" * 60)

[1] Logging in legitimately using Room101 credentials...
[2] Attempting to send a fake fire alert to hospital/Room102/sensors...
-----
[3] Check the Node-RED Dashboard now!
Did Room 102 get affected? (It should be completely safe and receive nothing).

[X] The broker kicked us out (Disconnected)! Reason: Normal disconnection - Unauthorized attempt!

(myenv) D:\Enhanced_Smart_Hospital>python IoT_Edge_Nodes_02/hack_test.py
Starting simulated hack test (Lateral Movement Attack)...
-----
[1] Logging in legitimately using Room101 credentials...
[2] Attempting to send a fake fire alert to hospital/Room102/sensors...
-----
[3] Check the Node-RED Dashboard now!
Did Room 102 get affected? (It should be completely safe and receive nothing).

[X] The broker kicked us out (Disconnected)! Reason: Normal disconnection - Unauthorized attempt!

(myenv) D:\Enhanced_Smart_Hospital>python IoT_Edge_Nodes_02/hack_test.py
Starting simulated hack test (Lateral Movement Attack)...
-----
[1] Logging in legitimately using Room101 credentials...
[2] Attempting to send a fake fire alert to hospital/Room102/sensors...
[X] The broker kicked us out (Disconnected)! Reason: Normal disconnection - Unauthorized attempt!

(myenv) D:\Enhanced_Smart_Hospital>
```

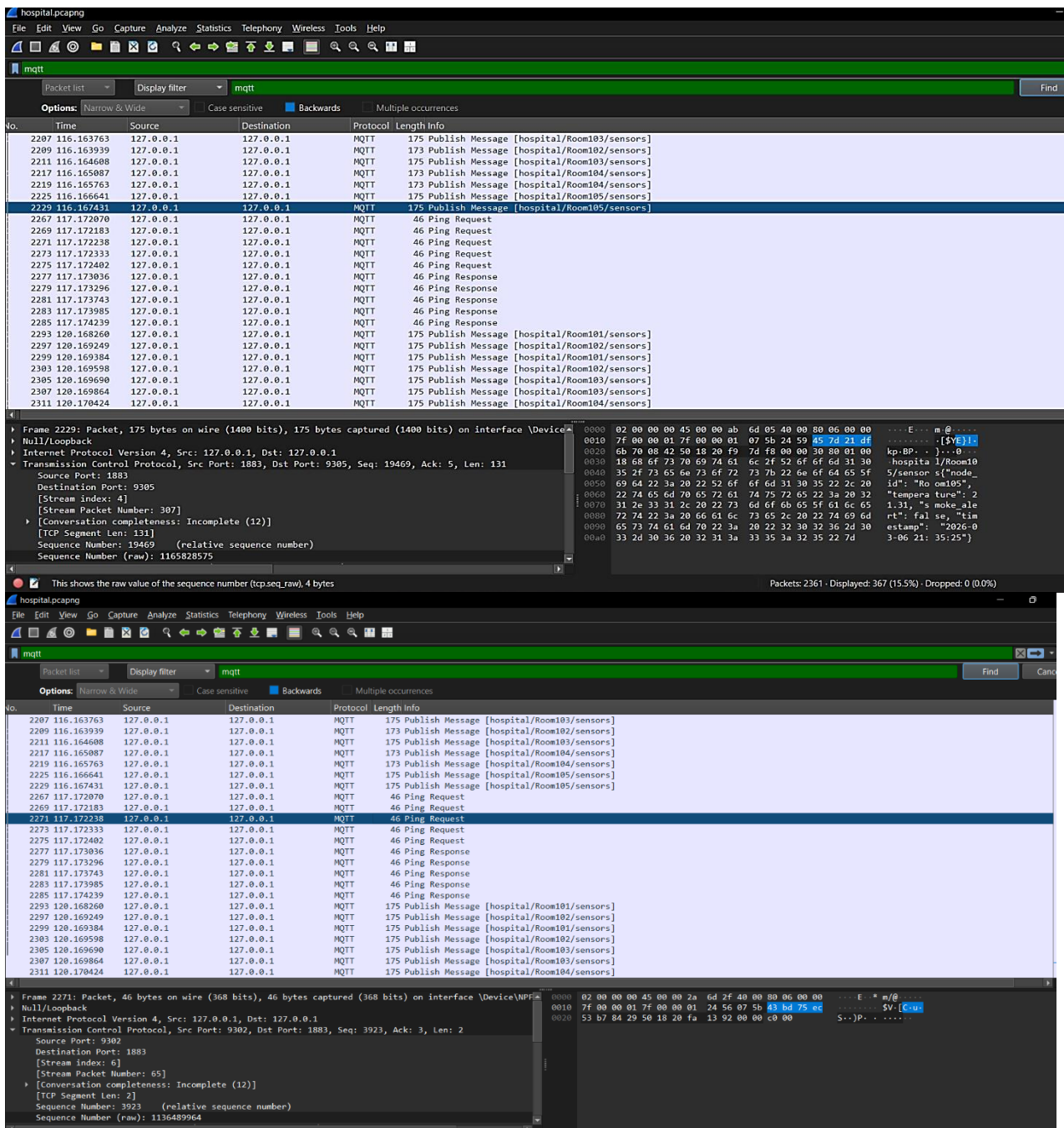
شكل (6): إحباط محاولة اختراق وهمية بواسطة جدار ACL الأمني.

8. تحليل الشبكة وتدفق البيانات (Network Packet Analysis)

تم استخدام برنامج Wireshark لتحليل حزم الشبكة (Packet Sniffing) على واجهة Loopback Adapter:

- تم رصد حزم الاتصال Connect Command للتأكد من إرسال الاعتمادات (Credentials).
- تم التقاط حزم Publish Message ومطابقة بيانات الحرارة داخل هيكل الـ TCP/IP.
- أثبت التحليل الشبكي أن محاولة الاختراق نتج عنها إرسال Ping Request فقط دون مرور أي حزم بيانات للغرفة المستهدفة، مما يمثل دليلاً جنائياً (Forensic Evidence) على صلابة النظام.

- برنامج Wireshark تظهر حزم ال MQTT وهي تعبر الشبكة.



شكل (7): تحليل حزم بروتوكول MQTT واصطياد محاولات الاتصال عبر Wireshark.

9. الخاتمة

يُظهر هذا المشروع نجاح تصميم وتنفيذ نظام "Enhanced Smart Hospital IoT Monitoring System" متكامل. لقد تمكنا من تجاوز مفهوم المراقبة البسيطة لبناء بنية تحتية آمنة تعتمد على بروتوكولات IoT القياسية مع تطبيق صارم لمعايير (Zero Trust). يثبت النظام قدرته ليس فقط على قراءة البيانات الحية وعرضها رسومياً، بل وحفظها تاريخياً، والأهم من ذلك، حمايتها من أي اختراقات داخلية أو تداخل، مما يجعله نموذجاً قابلاً للتطبيق في بيئات الرعاية الصحية الحساسة.

10. المراجع (References)

1. **Eclipse Foundation**. (n.d). *Eclipse Paho MQTT Python Client API Documentation*. Retrieved from <https://eclipse.dev/paho/index.php?page=clients/python/docs/index.php>
2. **Light, R**. (2017). *Mosquitto: An Open Source MQTT Broker*. Eclipse Foundation. Retrieved from <https://mosquitto.org/documentation>
3. **Node-RED**. (n.d). *Node-RED User Guide and Core Nodes API*. OpenJS Foundation. Retrieved from <https://nodered.org/docs>
4. **Connelly, S & ,Rose, S., Borchert, O., Mitchell, S**. (2020). *(NIST Zero Trust Architecture*. National Institute of Standards and Technology (Special Publication 800-207). <https://doi.org/10.6028/NIST.SP.800-207>
5. **Beale, J & ,Orebaugh, A., Ramirez, G**. (2006). *Ethereal Network Protocol & Wireshark*. Syngress Publishing. *Analyzer Toolkit*